Getting this on your iPad

You want a link you can open in Safari, add to your Home Screen, and use like an app. That takes about 20 minutes once, then never again.

The hosting is **Streamlit Community Cloud** — free, and it runs Python, which Netlify cannot. Netlify only serves static files, so it will not work for this one the way it did for the Command Center.

What you need

- A GitHub account (free)
- A Streamlit Community Cloud account (free, sign in with GitHub)
- Your Anthropic API key

You can do all of it from the iPad. A computer makes step 1 faster, and it is required if you want the Gmail feature.

Step 1 — Put the code on GitHub

From a computer:

```
cd afh_acquisition_pipeline
git init
git add .
git commit -m "AFH acquisition pipeline"
git branch -M main
git remote add origin https://github.com/YOURNAME/afh-pipeline.git
git push -u origin main
```

From the iPad: go to github.com in Safari, create a new **private** repository named `afh-pipeline`, then use **Add file → Upload files** and upload the project folder's contents.

Make the repository **private**. `.gitignore` already excludes `.env`, `credentials.json`, `token.json`, `.streamlit/secrets.toml` and `data/` — check that none of those got uploaded anyway.

Step 2 — Deploy

1. Go to https://share.streamlit.io and sign in with GitHub.
2. **Create app → Deploy a public app from a repo.** (The app itself will be password-protected in step 3; this setting is about the repo.)
3. Repository: your `afh-pipeline`. Branch: `main`. **Main file path:** **`streamlit_app.py`**
4. Deploy. First build takes a few minutes while it installs the packages.

You get a URL like `https://yourname-afh-pipeline.streamlit.app`.

Step 3 — Add your secrets

This step is not optional. Until you do it, the app will refuse to open.

In your app on share.streamlit.io: ⋮ **menu → Settings → Secrets.** Paste:

```
APP_PASSWORD = "pick something long you don't use elsewhere"
ANTHROPIC_API_KEY = "sk-ant-..."
ANTHROPIC_MODEL = "claude-sonnet-5"
```

Save. The app restarts on its own.

A Streamlit URL is effectively public — anyone who has it can open it. Without a password, a stranger could spend your API credits and generate letters of intent carrying your company name and signature block. That is why the app closes itself rather than running unprotected.

Step 4 — Add it to your Home Screen

1. Open the URL in **Safari** (not Chrome — Chrome on iOS cannot do this).
2. Tap the **Share** button.
3. **Add to Home Screen.** Name it something short like “AFH Screening”.

It now opens full screen with no browser bars, like the Command Center.

Step 5 — Fill in Settings

Open the app, go to the **Settings** tab, and enter:

- Your company legal name, signer, phone, email, mailing address. These go on the LOI letterhead, so use the exact name from your filed documents.
- Your own revenue and cost figures. Nothing is pre-filled and the operating model will show an incomplete-inputs error until you enter them.

Tap **Save settings**, then **Download a backup** and keep the file in your Files app. The hosting disk gets wiped whenever the app restarts or you push a code change, and the backup is how you get your work back.

Optional — connecting Gmail

Skip this unless you want the app pulling alerts by itself. Pasting alert text into the **Add** tab does the same job with no setup.

The Google sign-in flow opens a browser window on the machine running the code, which cannot work on a hosted server. So you run it once on a computer and move the resulting token into the app's secrets.

On a computer, with `credentials.json` in the project folder:

```
pip install -r requirements.txt
python -c "import gmail_ingestion; gmail_ingestion.get_gmail_service()"
```

A browser opens, you approve, and `token.json` appears in the folder.

Open `token.json`, copy the whole thing, and add it to your Streamlit secrets on one line inside single quotes:

```
GMAIL_TOKEN_JSON = '{"token": "...", "refresh_token": "...", "client_id": "...",
```

A **Check for new listing alerts** button then appears on the Add tab, and the **Put a draft in Gmail** button appears on each property.

One catch: while your Google Cloud consent screen is in Testing status, the refresh token expires about every 7 days and you repeat this. Publishing the consent screen stops that.

Running it on a computer instead

```
pip install -r requirements.txt
cp .streamlit/secrets.toml.example .streamlit/secrets.toml   # then edit it
streamlit run streamlit_app.py
```

Opens at `http://localhost:8501`. For a local-only run you can put `ALLOW_UNPROTECTED = "true"` in secrets instead of a password.

Things that will confuse you later

Your data disappeared. The hosting disk is temporary. Restore from a backup on the Settings tab. Download a fresh backup after any session that mattered.

No agent email. Listing alerts link to the listing but do not include the agent's inbox. Open the listing, find the agent, type the address into the field on the property page.

"Open in Mail" sends no attachment. `mailto:` links cannot carry files. Download the PDF first, then attach it from Files once Mail opens. Nothing is ever sent for you — not here, and not through the Gmail button, which only creates a draft.

A property shows RED before it was scored. That is the screening filter, not the AI: out of county, too few bedrooms, outside the price band, or no seller-motivation signal. Open it to see which.

The operating model shows an error. Your figures are missing on the Settings tab. That is deliberate — no reimbursement rate is filled in for you, because a guessed rate would flow into a rent number you sign for years.

Updating the app later

Push a change to GitHub and Streamlit redeploys within a minute or two. Take a backup first, because a redeploy wipes the data disk.